

Prompt para Cursor — Pipeline de Agentes para Crear Contenido Viral de TikTok

Pega esto en Cursor (como instrucción inicial o como Agent.md del proyecto).

Contexto

Quiero un sistema tipo pipeline en Python, mismo patrón que uso en mis otros proyectos (agentes en `src/agents/`, cada uno con una sola responsabilidad, orquestados en secuencia, con inputs/outputs en JSON).

Este pipeline recibe un tema o producto y genera el **guion completo de un video de TikTok listo para grabar y editar**: hook, desarrollo, cortes, textos en pantalla, CTA, loop final, audio sugerido, caption y hashtags. Cada agente lee un **skill** (`.md` con las reglas de esa parte) antes de generar su bloque, igual que en el pipeline de landings.

Estructura del proyecto

```
tiktok_pipeline/
├── src/
│   ├── agents/
│   │   ├── 01_trend_research.py
│   │   ├── 02_hook.py
│   │   ├── 03_script.py
│   │   ├── 04_pattern_interrupts.py
│   │   ├── 05_onscreen_text.py
│   │   ├── 06_cta.py
│   │   ├── 07_loop.py
│   │   ├── 08_audio.py
│   │   ├── 09_caption_hashtags.py
│   │   ├── 10_shotlist.py
│   │   └── 11_qa.py
│   ├── skills/
│   │   ├── hook_skill.md
│   │   ├── script_skill.md
│   │   ├── pattern_interrupt_skill.md
│   │   ├── onscreen_text_skill.md
│   │   ├── cta_skill.md
│   │   ├── loop_skill.md
│   │   ├── audio_skill.md
│   │   ├── caption_hashtags_skill.md
│   │   └── qa_checklist_skill.md
│   ├── pipeline.py
│   └── llm_client.py
├── output/{slug}/
│   ├── guion.json
│   ├── shotlist.md
│   └── qa_report.json
├── .env
└── logs/mejoras.json
```

Regla de oro: cada agente lee su skill antes de generar

Cada agente NO improvisa el estilo. Antes de llamar al LLM, carga el `.md` correspondiente de `src/skills/` y lo inyecta como parte del system prompt. El skill contiene: qué es obligatorio en esa parte, la regla de redacción, y un ejemplo corto. Así el resultado es consistente aunque cambie el tema.

Ejemplo de contenido de `hook_skill.md` (así deben ser TODOS los skills, mismo formato):

```
# Skill: Hook

## Regla
Los primeros 3 segundos deciden si siguen viendo. Sin intro, sin "hola
qué tal". Entra directo con una promesa, pregunta o afirmación fuerte.

## Obligatorio
- Máximo 1 frase, dicha en los primeros 3 segundos.
- Genera curiosidad o tensión, no describe el video.
- Nunca empieza con "hoy les voy a hablar de..." o saludo.

## Ejemplo
"Nadie te dice esto sobre ahorrar dinero"

## Output esperado (JSON)
{ "hook": "", "variante_2": "", "variante_3": "" }
```

Repite este mismo formato para cada parte, usando las reglas que ya validamos:

- **Script (desarrollo):** un solo mensaje central, no tres temas mezclados. Estructura tipo paso 1 / paso 2 / paso 3 o historia con giro.
- **Pattern interrupts:** marca cada 2-3 segundos dónde debería haber cambio de plano, zoom o corte, para que no sea un plano fijo.
- **Texto en pantalla:** palabra o frase clave por escena, sincronizada con lo que se dice, pensado para verse sin sonido.
- **CTA:** una sola acción concreta (comentar, guardar o seguir), nunca las tres a la vez.
- **Loop final:** la última frase debe conectar con la primera para que el video se repita solo.
- **Audio:** sugiere tipo de sonido (trend, voz en off, música de fondo) según el ritmo del guion, no un archivo específico.
- **Caption + hashtags:** el caption refuerza el hook sin repetirlo, 3 a 5 hashtags de nicho, no genéricos tipo #viral o #fyp.

`qa_checklist_skill.md` contiene el checklist de errores que matan retención (intro larga, sin texto en pantalla, plano fijo más de 4-5 segundos, CTA ausente o apurado al final). El agente 11 (QA) lo usa para puntuar el guion completo y marcar qué parte falla.

Flujo del pipeline

1. `01_trend_research.py` — Recibe el tema/nicho del usuario, investiga sonidos y formatos en tendencia (si hay acceso a web search o API), devuelve `trend.json` con sugerencias.
2. `02_hook.py` — Genera 3 variantes de hook usando `trend.json` + `hook_skill.md`.
3. `03_script.py` → `09_caption_hashtags.py` — Cada uno lee el output acumulado + su skill, genera su bloque, va armando `guion.json`.
4. `10_shotlist.py` — Convierte `guion.json` en un brief de producción legible: escena por escena, con tiempo aproximado, texto en pantalla, tipo de plano y nota de edición. Esto es lo que se usa para grabar.
5. `11_qa.py` — Revisa el guion completo contra `qa_checklist_skill.md`, devuelve `qa_report.json` con score y qué agente debe regenerar si algo falla.

Requisitos técnicos

- Cada agente es una función `def run(input: dict) -> dict` para poder testarlos sueltos o encadenarlos.
- `pipeline.py` orquesta todo en orden, loggea cada paso en `logs/mejoras.json` (igual que en mis otros pipelines).
- Usa el cliente LLM que ya tengo en `llm_client.py` (Claude API).
- Manejo de errores: si un agente falla, el pipeline debe poder reintentar solo ese agente sin re-correr todo desde cero.
- El output final (`shotlist.md`) tiene que ser algo que se pueda leer en el celular mientras grabo, no solo JSON crudo.

Qué quiero que hagas ahora

1. Crea la estructura de carpetas completa.

2. Escribe los 9 archivos de skills en `src/skills/` con el formato de ejemplo de arriba (regla + obligatorio + ejemplo + JSON esperado), usando las reglas de cada parte que te di.
3. Escribe `pipeline.py` y `llm_client.py`.
4. Escribe los agentes `01` a `11` siguiendo el contrato de input/output.
5. Al final, corre un ejemplo end-to-end con un tema de prueba y muéstrame el `shotlist.md` generado.